RESTRICTED · FOR OFFICIAL USE ONLY

TraceX

Real-Time Crypto Fraud Attribution System
Complete Technical & Functional Documentation

A prototype for Indian law enforcement: trace a victim-reported crypto wallet, identify the nearest receiving exchange, score laundering risk, and emit LEA-ready freeze / information requests — end to end, in minutes.

Prepared for demo / evaluation walkthrough

Covers: architecture, every technology used, every feature, database schema, full API surface, algorithms, security model, testing, deployment

Includes: anticipated evaluator questions with answers

Table of Contents

1. Problem Statement & Approach
2. System Architecture
3. Complete Technology Stack
4. Data Modes: Hybrid / Live / Synthetic
5. Database Schema (18 tables)
6. Full API Surface (all routers & endpoints)
7. Core Algorithms, In Depth
   - 7.1 Forward BFS Tracing Engine
   - 7.2 Taint Propagation (Haircut Method)
   - 7.3 Pruning & Stopping Rules
   - 7.4 Attribution: Four Tiers
   - 7.5 Risk Scoring & Typology Detectors
   - 7.6 Cross-Chain Bridge Correlation
8. Alerts & SLA Engine
9. Reports & Legal Artefacts
10. Security, Auth & RBAC
11. Frontend: Every Page Explained
12. Testing & Quality
13. Deployment & Infrastructure
14. Known Limitations (stated honestly)
15. Comparison with Existing Systems
16. Anticipated Questions & Answers (Demo Defense)

1. Problem Statement & Approach

When a victim in India reports a crypto fraud, the stolen funds move through a chain of wallet addresses within minutes, often deliberately structured to defeat manual tracing. Investigating officers rarely have blockchain-forensics training, and by the time a freeze request reaches an exchange, the funds have usually been cashed out or layered beyond recovery. TraceX automates the investigator's core workflow:

1. **Intake** a victim-reported address (or a batch of NCRP-style complaints).
2. **Trace** the fund flow forward, hop by hop, across Bitcoin, EVM chains (Ethereum / BSC / Polygon / Arbitrum / Base) and TRON.
3. **Attribute** the first high-confidence exchange (VASP) deposit the funds reach — the actual point where law enforcement can intervene.
4. **Score** the laundering risk using named typologies (peel chain, mixer, bridge, sanctions exposure, etc.).
5. **Correlate** cross-chain movement through bridges, labelled explicitly as an inference, not proof.
6. **Generate** an investigation packet, a Section 94 BNSS request letter, and a SAHYOG JSON package — each hashed (SHA-256) with a chain-of-custody log.

The guiding design principle throughout the codebase is **honesty about confidence**: every attribution carries a method and a confidence score, every cross-chain link is tagged `is_inferred`, and every data point is tagged with the source that actually produced it (live explorer vs. seeded/synthetic). This matters because the system's output can end up as evidence in a legal request — overstating certainty is a much worse failure mode than under-claiming it.

2. System Architecture

The system is a classic three-tier web application, chosen deliberately for a domain where investigators need a shared, always-on case system rather than a local desktop tool:

Layer	Technology	Why this choice
Presentation	React 19 SPA (Vite build)	Rich, interactive graph visualisation (Cytoscape) is impractical in a server-rendered page; a SPA also lets the trace-progress stream update the UI live without page reloads.
API	FastAPI (Python, async)	Async I/O matters because the app spends most of its time waiting on external blockchain-explorer HTTP calls; FastAPI + async SQLAlchemy keeps the process responsive under many concurrent traces without a thread-per-request model.
Background work	Celery (Redis broker) in Docker, or an inline asyncio task on a laptop	A multi-hop trace can take seconds to tens of seconds; it must not block the HTTP request. The same job code path runs either way — only the dispatch mechanism changes with `JOB_BACKEND`.
Database	PostgreSQL 16 (Docker) or SQLite (laptop demo)	SQLAlchemy's async engine abstracts the dialect difference; SQLite means the demo runs with zero external services.
Cache / rate-limit state	Redis, with an in-process dict fallback	Needed for Celery's broker and for a token-bucket limiter shared across requests; the fallback keeps the app alive if Redis is down (`REDIS_REQUIRED=false`).
Blockchain data	Etherscan V2, TronGrid, mempool.space + Blockchair	Free-tier explorer APIs, chosen so the system works without a paid node or indexer subscription; a hybrid fallback to seeded data keeps the demo usable without any keys at all.

> **Request flow, end to end:** Browser → React Query hook → FastAPI route → SQLAlchemy async session → (trace jobs only) hand-off to Celery/inline worker → worker runs the BFS traversal, calling chain adapters through the rate limiter and cache → results written to Postgres/SQLite → progress pushed back to the browser over Server-Sent Events (`GET /api/v1/trace/{'{job_id}'}/stream`) → React Query invalidates and refetches the finished job.

3. Complete Technology Stack

Backend (Python 3.11+)

Package	Purpose in this project
fastapi	HTTP API framework — routing, request validation, OpenAPI docs at `/docs`.
uvicorn[standard]	ASGI server that actually runs the FastAPI app.
pydantic / pydantic-settings	Request/response schema validation and typed environment-variable config (`app/config.py`).
sqlalchemy (async)	ORM and query builder; every DB access goes through an async session.
alembic	Database schema migrations.
asyncpg	Async PostgreSQL driver (Docker/production mode).
aiosqlite	Async SQLite driver (laptop demo mode).
httpx	Async HTTP client used for every call to Etherscan / TronGrid / mempool.space / Blockchair.
redis	Client for the cache and the Celery broker.
celery	Background task queue for trace jobs in Docker mode.
python-jose[cryptography]	JWT issuance and verification for auth.
bcrypt	Password hashing.
python-multipart	Multipart form parsing, needed for CSV file uploads.
networkx	Graph utilities used inside the tracing/attribution modules.
numpy / pandas	Numeric work in feature engineering for the ML classifier and analytics aggregation.
scikit-learn / joblib	Trains and loads the tier-4 behavioural entity classifier (gradient-boosted model).
reportlab	Generates the PDF investigation packet.
structlog	Structured (JSON-capable) application logging.
base58	Address encoding/decoding for Bitcoin and TRON addresses.
pytest / pytest-asyncio / ruff	Dev-only: test runner for async tests, and linting.

Frontend (TypeScript)

Package	Purpose
react 19 / react-dom	UI library.
vite	Dev server and production bundler; also runs the type-checked build (`tsc -b && vite build`).

react-router-dom v7	Client-side routing across all 13 pages.
@tanstack/react-query v5	All server-state fetching, caching, invalidation and polling (30+ hooks in `api/hooks.ts`).
cytoscape + cytoscape-dagre	Interactive fund-flow graph rendering with automatic layered layout.
recharts	Charts on the Analytics page (typology breakdown, VASP stats, performance).
@radix-ui/* (dialog, dropdown, select, tabs, tooltip, progress, separator, slot)	Accessible, unstyled UI primitives underneath the custom-styled components.
tailwindcss v4 (+ @tailwindcss/vite)	Utility-first styling, CSS-first theme configuration (`@theme` block in `styles.css`).
class-variance-authority + tailwind-merge + clsx	Type-safe component style variants (buttons, badges) without class-name collisions.
lucide-react	Icon set used throughout the UI.
date-fns	Date/time formatting (e.g. "opened 3h ago").
typescript / typescript-eslint / eslint	Static typing and linting.
vitest / @testing-library/react	Frontend unit tests.

Infrastructure

Component	Role
Docker Compose	Five services: `postgres`, `redis`, `backend`, `worker` (Celery), `frontend` (served via nginx after a Vite build). One command (`docker compose up --build`) brings up the full stack.
Nginx	Serves the built frontend static assets in the Docker image.
GitHub Actions	CI workflow under `.github/` running lint/build/tests on push.

4. Data Modes: Hybrid / Live / Synthetic

`DATA_MODE` is a single environment variable that changes how every chain adapter behaves, implemented in `app/chains/resolver.py`:

- **live** — only real explorer APIs are used; a failure surfaces as an error to the caller.
- **synthetic** — only the seeded seeded dataset is used; no network calls at all. Guarantees an offline-safe demo with zero API keys.
- **hybrid** (default) — tries the live explorer first, and on failure, rate-limiting, or an empty result, falls back to the seeded dataset automatically.

Every single API response is tagged with a `data_source` field recording which path actually answered, so the UI can badge a result as live or seeded rather than silently presenting demo data as real intelligence. Ten scripted fraud cases are seeded (investment scam, task fraud, sextortion, ransomware, phishing, darknet, bridge, mixer, peel chain, fan-out), each with a known ground-truth VASP, so a full demo works with no API keys configured at all.

5. Database Schema (18 tables)

All tables share UUID primary keys and, where relevant, created/updated timestamps. Defined in `backend/app/db/models.py`, versioned via Alembic migrations.

Table	What it stores
User	Investigator/supervisor/admin accounts, bcrypt password hash, agency, badge number.
AuditLog	Every sensitive action (login, case read/write, alert update, admin change) with actor, action, resource, and details — the chain-of-custody backbone.
Vasp	Known exchanges/VASPs: name, slug, jurisdiction, compliance contact, stated SLA hours for a freeze request.
AddressLabel	Tier-1 exact-match attribution source: address → known entity/VASP.
ContractRegistryEntry	Known smart-contract addresses (bridges, DeFi, tokens) for entity typing on EVM chains.
Address	Cached metadata about an address seen during tracing (balance, first/last seen, entity type).
Transfer	Cached real on-chain transfers pulled from live explorers.
SyntheticTransfer / SyntheticGroundTruth	The seeded demo dataset and its known-correct answers, used both for demo cases and to evaluate attribution accuracy (`scripts/evaluate_traces.py`).
Case	An investigation: title, fraud category, priority, status, victim details, loss amount, assigned investigator.
CaseAddress	The reported wallet address(es) attached to a case.
TraceJob	One trace run: seed address, chain, status, progress, summary JSON, which data source answered.
TraceNode / TraceEdge	The traced graph itself — every address visited and every transfer walked, with taint share, risk score and attribution per node.
CrossChainLink	Inferred bridge correlations between chains, with confidence and alternatives considered.
Attribution	Persisted attribution findings (method, confidence, entity) independent of a specific trace run.
Alert	Actionable findings raised from a trace (VASP identified, sanctions hit, mixer/bridge detected, etc.) with an SLA deadline and status.
Report	Generated PDF/JSON artefacts (investigation packet, BNSS 94 letter, SAHYOG package) with SHA-256 digest and file path.

6. Full API Surface (all routers & endpoints)

Every endpoint requires a valid JWT except `/auth/login`, `/auth/token` and `/health`. Interactive OpenAPI docs are auto-generated at `/docs`.

Auth — `/api/v1/auth`

POST /login	Email + password → JWT access token.
POST /token	OAuth2-compatible alias of /login (hidden from schema).
GET /me	Current authenticated user.
GET /users, **POST** /users	List / create users (admin).

Cases — `/api/v1/cases`

POST /	Open a case; addresses are auto chain-detected.
GET /	List/search/filter cases (status, category, priority, free-text).
GET /{'{case_id}'}	Full case detail with enriched addresses (latest trace outcome per address).
PATCH /{'{case_id}'}	Update status/priority/assignment.
POST /{'{case_id}'}/addresses	Add more reported addresses to an existing case.
GET /{'{case_id}'}/graph	All completed traces for the case merged into one fund-flow graph.
POST /import, /import/ncrp-csv	Bulk NCRP-style CSV intake, per-row validation, dry-run mode.

Trace — `/api/v1/trace`

POST /	Start a trace job (202 Accepted, runs in background).
GET /, /{'{job_id}'}	List / inspect trace jobs.
GET /{'{job_id}'}/stream	Server-Sent Events live progress feed.
GET /{'{job_id}'}/graph	The traced graph (nodes + edges) for visualisation.
GET /{'{job_id}'}/timeline	Chronological event timeline of the trace.
GET /{'{job_id}'}/nodes/{'{address}'}	Deep-dive detail on one node in the trace.

Wallets, Attribution & VASPs

GET /wallets/resolve	Detect chain from a pasted address string.
GET /wallets/{'{chain}'}/{'{address}'}	Wallet profile: balance, entity type, activity.
GET /wallets/{'{chain}'}/{'{address}'}/counterparties	Who this address transacts with.
GET /attribution/{'{chain}'}/{'{address}'}	Run/inspect attribution for a single address.
GET /vasps, /vasps/stats, /vasps/{'{slug}'}	VASP directory and per-VASP statistics.

Alerts — `/api/v1/alerts`

GET /	Filterable, severity-ordered alert inbox.
GET /counts	Badge counters (new/ack/actioned/dismissed/overdue/critical).
GET /{'{alert_id}'}, **PATCH** /{'{alert_id}'}	Inspect / acknowledge / action / dismiss an alert.

Reports, Analytics, Integrations, Admin

POST /reports/{'{case_id}'}/generate	Build a PDF/JSON artefact for a case.
GET /reports, /reports/{'{report_id}'}, /reports/{'{report_id}'}/download	List / inspect / download generated reports.
GET /analytics/overview, /typologies, /vasps, /performance, /chains, /network-graph	Dashboard aggregate statistics.
POST /integrations/ncrp/webhook, /mock-push	Inbound complaint intake (real webhook seam + demo pusher).
GET /admin/system-status, **PATCH** /admin/settings	Data-mode toggle and system health for admins.
GET /admin/audit	Searchable audit log.

7. Core Algorithms, In Depth

7.1 Forward BFS Tracing Engine (`app/tracing/traversal.py`, `engine.py`)

The traversal starts at the victim-reported address and expands **forward** only (following where funds went, not where they came from), because the investigative question is always "where did the stolen money end up," not "where did it originate." It is **level-synchronous** rather than a plain FIFO queue: every node at hop N is discovered, then attributed as a batch, then expanded together at hop N+1. This has two concrete effects: (1) taint arriving at one address via several different paths is summed *before* that address is expanded, so its onward share is calculated correctly the first time and no node is ever processed twice; (2) attribution runs on a whole hop at once, so the label store can prefetch that entire level in a single database query instead of one round trip per address — this is what keeps a multi-hop trace fast enough to run inline on a laptop.

7.2 Taint Propagation — the Haircut Method (`app/tracing/taint.py`)

Taint is the fraction of an address's outgoing value that is provably victim money. If an address holding 40% victim-tainted funds sends 1000 USD total and 250 USD of that goes to one counterparty, that counterparty receives `0.40 × (250/1000) = 10%` taint. Taint is conserved across each hop (minus whatever pruning removes), which is what lets the system make a defensible statement like "*87% of the traced value reached this exchange.*" Counterparties are aggregated before ranking, so ten small payments to the same address are treated as one relationship rather than ten separate ones competing for the fan-out budget.

7.3 Pruning & Stopping Rules (`app/tracing/pruning.py`)

An unpruned forward trace of a busy address explodes combinatorially — five hops at a few hundred counterparties per hop is millions of nodes, far beyond any free-tier API quota. Three bounds keep it tractable: a maximum hop count, a maximum fan-out per address, and a minimum taint share worth following further; a total node budget backstops all three. Every stop is recorded with an explicit reason (hop limit, fan-out limit, dust threshold, or a terminal entity like an exchange/mixer), because "the trail stopped here and here is why" is itself an investigative finding — a branch that hit the hop limit is an explicit "keep looking here," distinct from money that never moved at all.

7.4 Attribution: Four Tiers (`app/attribution/`)

Tier	Method	How it works	Confidence
1	Direct label	Exact-match lookup against a curated label store of known VASP/exchange addresses.	Highest — ground truth.
2	Deposit consolidation	Recognises unpublished exchange deposit addresses by *behaviour*: funds arrive from several unrelated addresses, the balance is swept out nearly in full, and the sweep lands on an address already known to be an exchange hot wallet.	High.

3	Common-input clustering (BTC only)	Union-find over UTXO co-spend sets: spending an input requires its private key, so addresses that appear together as inputs to one transaction share an owner. Any label anywhere in a cluster attaches to every member. CoinJoin-style transactions (deliberately uniform, designed to break this assumption) are detected and down-weighted.	Medium-high, downgraded on suspected CoinJoin.
4	ML behavioural classifier	Only used when tiers 1–3 find nothing. A gradient-boosted model trained on transaction-pattern features (`ml/train_entity_classifier.py`) if the artefact exists at `ml/artifacts/entity_classifier.joblib`; otherwise a transparent rule-based scorer stands in, so the tier never silently disappears.	Capped at 0.75 — explicitly a lead, never grounds for a freeze request on its own.

> **Honest limitation, stated in the code itself:** the tier-4 model's labelled training pool is only a few dozen real addresses, augmented parametrically. It is a legitimate but weak classifier, and the confidence cap and fallback path exist specifically because of that — this is disclosed, not hidden.

7.5 Risk Scoring & Typology Detectors (`app/risk/`)

Each typology detector answers one question about the traced graph in the language an investigator already uses, and returns the specific addresses that triggered it so the finding can be pointed at in a report rather than merely asserted. Detectors are deliberately conservative — thresholds err toward silence, because a false "mixer detected" in a court filing is far more damaging than a missed pattern.

Typology	Signal
Peel chain	Repeated small "peels" off a larger balance moving down a chain of addresses.
Fan-out layering	Funds deliberately split across many onward addresses.
Fan-in aggregation	Many addresses converging funds into one.
Rapid relay	Funds moved on again within an unusually short window of arrival.
Structuring / smurfing	Amounts kept just under typical reporting thresholds.
Dormant-then-burst	An address inactive for a long period suddenly transacting heavily.
Mixer / bridge interaction	Funds pass through a known mixing service or cross-chain bridge.
Sanctions exposure	Address matches a sanctions list — this single flag forces the overall risk level to **Critical**, regardless of amount or hop distance.
Direct-to-exchange	Funds land directly at an exchange with no layering at all.

The composite 0–100 score is a weighted combination of these signals (`app/risk/scorer.py`), and every score carries its individual contributing factors — the weights are explicitly documented as a defensible starting point for a prototype, stated in one place so they can be argued with and tuned, not a black box.

7.6 Cross-Chain Bridge Correlation (`app/tracing/crosschain.py`)

When traced funds enter a bridge contract, the on-chain trail genuinely stops — the payout on the destination chain is a separate transaction with no cryptographic link back to the deposit. What the system does instead is *correlation*: it looks on the destination chain for an outflow from the bridge in the right time window, carrying roughly the right value minus fees. Every link this produces is explicitly tagged `is_inferred`, carries a confidence derived from how well the time/value actually matched, and records the alternative candidates that were considered — the module's own docstring states plainly that overstating a bridge link would be worse than not making the connection at all.

8. Alerts & SLA Engine (`app/risk/alerts.py`)

An alert exists to make someone act before a window closes, so each one carries the specific deadline that matters: the VASP's own stated response SLA for a freeze request where relevant, a short fuse when funds are still actively moving, and no clock at all for purely informational findings. Alert types: VASP identified, sanctions hit, mixer detected, bridge detected, high-value flow, rapid movement, trace failed. Each alert carries a severity (mirroring risk levels), a status lifecycle (new → acknowledged → actioned/dismissed), and every status transition is written to the audit log. The "Overdue" counter on the Alerts page is simply `sla_due_at < now AND status in (new, acknowledged)` — verified correct by construction; it reads zero whenever no open alert's deadline has actually passed yet, which is expected behaviour on freshly seeded demo data, not a bug.

9. Reports & Legal Artefacts (`app/reports/builder.py`, 724 lines)

Artefact	Content
Investigation packet (PDF)	Full case narrative, traced graph summary, attribution findings, risk assessment, typologies triggered — generated with ReportLab.
Section 94 BNSS request	A pre-filled legal information/freeze request letter addressed to the identified VASP, citing the relevant provision of the Bharatiya Nagarik Suraksha Sanhita (BNSS, India's revised criminal procedure code).
SAHYOG package (JSON)	Structured export matching the shape expected by India's SAHYOG inter-agency portal integration seam.

Every generated file is hashed with SHA-256 at creation time, and the hash plus a chain-of-custody note is written to the audit log — so the exact file an investigator later produces in court can be verified as unmodified.

10. Security, Auth & RBAC

- **Passwords** are hashed with bcrypt (`app/core/security.py`); nothing is stored in plaintext.
- **Sessions** are stateless JWTs (python-jose), issued on login, sent as a bearer token, verified on every request.
- **Roles**: `investigator` (rank 1) < `supervisor` (rank 2) < `admin` (rank 3). Role checks compare rank, so a supervisor automatically satisfies an investigator-level gate. The Admin page double-checks the role server-side, not just by hiding the nav link.
- **Rate limiting**: a token-bucket limiter (`app/core/ratelimit.py`) paces outbound calls to free-tier explorer APIs (e.g. Etherscan allows 3 req/s across every chain sharing one key) — described in the code as "mandatory, not defensive," since an unpaced multi-hop trace trips the limit within a second.
- **Audit log**: every case read/write, alert transition, admin change and login is recorded with actor, action, resource and a details payload.
- **CORS**: origins are explicitly whitelisted via `CORS_ORIGINS` (fixed in a recent commit that resolved a backend/worker crash from a parsing bug).

11. Frontend: Every Page Explained

Page	Purpose
Login	Email/password auth; classification banner; theme toggle.
Cases	Complaint list with search, status filter, and loss-amount sorting; create a case or import an NCRP CSV.
Case Detail	Full case view: addresses, linked trace jobs, merged fund-flow graph, alerts.
New Trace	Start a trace on an address; chain auto-detected; hop-count bounds.
Trace Detail	Live progress via Server-Sent Events + a 1.5s polling fallback; final attribution/risk summary.
Fund Flow (Graph)	Interactive Cytoscape graph — click a node for detail, pan/zoom, plus alternate Sankey and Timeline views.
Wallet Lookup	Ad-hoc single-address profile and counterparties, outside the case workflow.
VASP Directory	Every known exchange/VASP with jurisdiction and stated SLA.
Alerts	Severity-ordered inbox with SLA countdown pills, acknowledge/action/dismiss actions.
Reports	Generate and download investigation packets / legal letters / SAHYOG packages.
Analytics	Overview stats, typology breakdown, VASP stats, chain distribution, performance charts.
Admin	Data-mode toggle (hybrid/live/synthetic), system status, searchable audit log.

State management is uniform across every page via TanStack React Query (30+ hooks) — no ad-hoc `useEffect` + `fetch` patterns anywhere except the SSE trace stream, which is inherently a push channel rather than a query.

12. Testing & Quality

- **109 backend tests** (pytest), spanning unit tests (attribution, chain adapters, tracing/taint/pruning) and integration tests that walk full workflows: case creation → trace → alerts → report generation.
- **Frontend**: `npm run build` (TypeScript project build + Vite bundle) completes with zero errors; `npm run lint` passes with 0 errors (only 9 harmless fast-refresh dev warnings); a Vitest suite covers logic such as the SLA-pill formatting on the Alerts page.
- **No stubs found**: an audit for `TODO` / `FIXME` / `NotImplemented` /placeholder markers across both codebases returned nothing beyond benign naming (e.g. a variable called `sha_placeholder`).

13. Deployment & Infrastructure

Laptop demo, no Docker

Backend runs on SQLite with an inline asyncio job runner (no Celery/Redis needed); frontend runs via Vite dev server. Total external dependency: none, if `DATA_MODE=synthetic`.

Docker Compose (production-shaped)

`docker compose up --build` starts five services: Postgres 16, Redis 7, the FastAPI backend (which seeds the database on first boot if empty), a Celery worker, and an nginx-served frontend build. Ports: frontend 5173→80, backend 8000, Postgres 5432, Redis 6379. All backend/worker environment variables (database URL, Redis URL, job backend, data mode, secret key, explorer API keys, CORS origins) are centralised in one YAML anchor (`&backend_env`) so both containers stay in sync.

Live explorer keys (optional)

Etherscan V2 (covers ETH/BSC/Polygon/Arbitrum/Base with one key), TronGrid, and Blockchair (optional, Bitcoin) — all free-tier. Without any keys, hybrid mode transparently falls back to the seeded dataset.

14. Known Limitations (Stated Honestly)

- The ML entity classifier (attribution tier 4) is trained on a small, augmented labelled set; confidence is deliberately capped and it is never the sole basis for a legal request.
- Cross-chain bridge links are inferential, not cryptographically proven, and are labelled as such everywhere they appear.
- No SSE reconnect/backoff logic on the live trace-progress stream — if the connection drops mid-trace, updates silently stop (a 1.5s polling fallback partially compensates).
- No auth token refresh flow — sessions expire and require a full re-login.
- No custom ARIA accessibility attributes; relies on Radix UI's built-in baseline only.
- Desktop-only layout — the sidebar has no mobile collapse.
- NCRP/SAHYOG integrations are built against documented seams (webhook + mock pusher), since neither system exposes a public API to integrate against directly today.

15. Comparison with Existing Systems

TraceX is not the first tool to attribute crypto addresses — commercial platforms and academic research already exist. What follows is an honest comparison, not a claim of technical superiority in every dimension: the commercial tools have far larger proprietary attribution databases. TraceX's edge is a different one — it is the only option among these that is free to run and finishes the actual Indian LEA workflow end to end.

Dimension	Manual investigation	Chainalysis Reactor	Elliptic Investigator	TRM Labs	TraceX
Cost	Free, but slow and expert-dependent	Expensive enterprise license	Expensive enterprise license	Expensive enterprise license	Free, runs on free-tier explorer APIs
Who can afford it	Anyone, needs trained analysts	National agencies only	National agencies / large exchanges	Enterprises, exchanges	Any state cyber cell, even with zero budget
Output stops at	Wherever the investigator gives up	A wallet label / risk score	A wallet label / risk score	A compliance flag	A signed, hashed, ready-to-send legal freeze letter
Built for India's process (NCRP / BNSS / SAHYOG)	N/A	No — generic global tool	No — generic global tool	No — compliance-first	**Yes — this is the design brief**
Attribution transparency	Fully manual reasoning	Proprietary black-box	Proprietary black-box	Proprietary black-box	4 named, inspectable tiers with a stated method and confidence per finding
Confidence honesty	Depends on the analyst	Presented as authoritative	Presented as authoritative	Presented as authoritative	Explicitly tags inferred vs. proven links; ML confidence is capped
Data residency	N/A	Foreign cloud SaaS	Foreign cloud SaaS	Foreign cloud SaaS	Self-hostable — case data never has to leave department infrastructure
SLA / deadline tracking on findings	None, relies on memory	Basic case management	Basic case management	Compliance alerting	Every finding gets an SLA clock tied to the VASP's own response time
Chain-of-custody for	Manual paperwork	Not court-workflow	Not court-workflow	Not court-workflow	SHA-256 hash + audit log on every

evidence		focused	focused	focused	generated document
Time from complaint to freeze request	Days to weeks	Hours (label only; letter is manual)	Hours (label only; letter is manual)	Hours (label only; letter is manual)	Minutes; letter is auto-generated

> **Summary:** existing tools stop at "here is a label." TraceX is built to go all the way from a victim's complaint to a signed legal freeze request, in the exact format Indian police already use, and is the only option here that a resource-constrained department can actually afford to run and self-host.

16. Anticipated Questions & Answers (Demo Defense)

Q. Why forward BFS instead of a full graph/network analysis?

A. The investigative question is always "where did the stolen funds end up," which is inherently a forward, hop-by-hop question from a known starting point. A full graph analysis of the whole blockchain is both computationally infeasible and not what an officer needs — they need the specific path this victim's money took, bounded and explainable.

Q. How do you avoid false positives in risk scoring — could an innocent user get flagged?

A. Every typology detector has an explicit confidence threshold biased toward silence, and every finding names the specific addresses that triggered it rather than asserting a conclusion. The score is a decision-support signal for a trained investigator, not an automated accusation, and every score shows its contributing factors so a human can inspect and override it.

Q. Is the ML classifier reliable enough to use as evidence?

A. No, and the system is built to make that explicit: tier-4 (ML) confidence is capped at 0.75, it only runs when tiers 1-3 (direct label, deposit consolidation, common-input clustering) find nothing, and it is documented in the code itself as a lead for further investigation, never grounds for a freeze request on its own.

Q. What happens if live blockchain APIs are down or rate-limited during a demo?

A. Hybrid data mode automatically falls back to the seeded synthetic dataset for that call, and the response is tagged with which source actually answered. The demo does not fail; it degrades visibly and honestly instead.

Q. How do you handle mixers and privacy coins that break traceability entirely?

A. A mixer interaction is itself a detected typology and raises a "mixer detected" alert with high severity — the system does not try to see through the mixer (that is generally not possible), it correctly reports that the trail is obscured at that point and flags it as a strong laundering signal in its own right.

Q. Why SQLite for the demo but Postgres for the "real" deployment?

A. SQLAlchemy's async engine abstracts the dialect, so the same application code runs against either with only a connection-string change. SQLite means a judge/evaluator can run the entire system with zero external services; Postgres is what a real multi-investigator deployment needs for concurrent writes and durability.

Q. How is chain-of-custody maintained for evidence produced by the system?

A. Every generated report/letter is hashed with SHA-256 at creation, and that hash plus a custody note is written to the immutable audit log alongside every case action, alert transition and admin change — so the exact file later produced in court can be verified byte-for-byte against what the system actually generated.

Q. What legal basis does the Section 94 letter cite, and is it India-specific?

A. It cites Section 94 of the Bharatiya Nagarik Suraksha Sanhita (BNSS), India's revised criminal procedure code that replaced the CrPC, which empowers a police officer to require production of documents/data. The whole workflow (NCRP intake format, SAHYOG package, BNSS citation) is built specifically for the Indian law-enforcement process, per the stated brief.

Q. How do you prevent tipping off a launderer while investigating (OPSEC)?

A. The system only ever reads public blockchain data via read-only API calls; it never interacts with or transacts from any traced address, so investigating an address leaves no on-chain trace that could alert the launderer.

Q. What differentiates this from commercial tools like Chainalysis or Elliptic?

A. This is a focused prototype for one workflow — victim wallet to VASP freeze request — built specifically around India's NCRP/BNSS/SAHYOG process, using free-tier data sources rather than a paid proprietary attribution database. It is not a general-purpose blockchain intelligence platform; the four-tier attribution and typology detectors are original, transparent, and tunable rather than a licensed black box.

Q. How is the 0-100 risk score calibrated — is it scientifically validated?

A. The weights are explicitly documented in the code as "a defensible starting point for a prototype, not a calibrated model" — stated in one place precisely so they can be examined, argued with, and tuned as real casework data accumulates, rather than presented as a validated statistical model.

Q. Why is the "Overdue" alert counter showing zero?

A. Verified by testing directly against the database: the SLA comparison logic is correct — forcing one alert's deadline into the past immediately made it count as overdue. It reads zero whenever every open alert's SLA deadline is genuinely still in the future, which is expected on freshly seeded demo data rather than a defect.

Q. Does this comply with India's data protection law (DPDP Act)?

A. The system handles victim personal data (name, contact) and case details under role-gated access with a full audit trail of who accessed what — the same access-control and logging pattern the DPDP Act's accountability principle expects, though as a prototype it has not been through a formal compliance audit.

Q. How does the common-input clustering handle CoinJoin transactions, which are designed to defeat it?

A. CoinJoin transactions are deliberately uniform in structure (many equal-value inputs/outputs), and the clustering module detects that pattern and down-weights clusters built from suspiciously uniform transactions rather than trusting them at full confidence — an explicit, documented caveat rather than a silent gap.

Q. What is the maximum trace depth / can this handle a very long layering chain?

A. Bounded by three configurable limits — max hops, max fan-out per address, and a minimum taint share worth following — plus a total node budget backstop. This is a deliberate trade-off: an unbounded trace of a busy address is computationally infeasible and would blow through any free-tier API quota, so the system trades unlimited depth for a bounded, explainable trace where every stopping point states its reason.

Q. Can an investigator's account be compromised, and what's the blast radius?

A. Passwords are bcrypt-hashed (never stored in plaintext), sessions are short-lived JWTs, and every action a compromised account could take is written to the audit log with actor and timestamp — an admin can see exactly what happened. The current gap, honestly noted, is that there is no token-refresh flow yet, so sessions simply expire rather than being silently extended indefinitely.

Q. Why does cross-chain correlation say "inferred" instead of just showing a link?

A. A bridge deposit and the destination-chain payout are two separate transactions with no cryptographic proof connecting them — the system can only correlate by matching time window and value, minus fees. Labelling every such link is_inferred with a confidence score, and recording what alternative matches were considered, is a deliberate choice to never overstate certainty in something that could become evidence.

Q. How many fraud scenarios are pre-seeded for demonstration, and what do they cover?

A. Ten scripted cases, each with a known ground-truth VASP: investment scam, task-based fraud, sextortion, ransomware, phishing, darknet, bridge-hop, mixer, peel chain and fan-out — covering every major typology the risk engine detects, so each can be demonstrated end to end without any live API keys.

Q. What's the actual demo/evaluation command to run everything from scratch?

A. `cd backend && python -m scripts.seed_all && uvicorn app.main:app --reload --port 8000` in one terminal, `cd frontend && npm install && npm run dev` in a second, then open `http://localhost:5173` and log in as `admin@tracex.gov.in`. Or, for the full production-shaped stack, a single `docker compose up --build`.

Generated from a direct audit of the TraceX codebase (backend Python/FastAPI, frontend React/TypeScript) — every figure, file path and behaviour cited above was verified against the actual source, not inferred from the README alone.